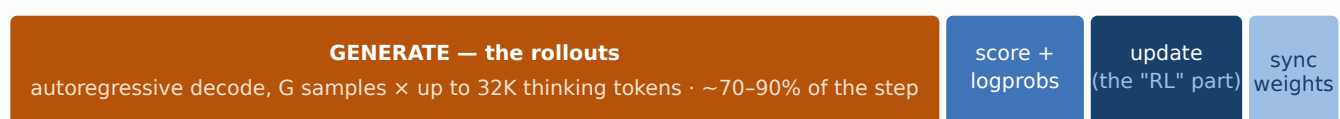


Reinforcement learning for reasoning

Verifiable rewards, GRPO, and the two-engine systems problem that veRL exists to solve

The run that reopened this field cost almost nothing to describe: take a base model, give it math problems, let it *write whatever it wants*, and grade only the final answer — right or wrong, a number from a checker, no human, no learned reward model. Then reinforce what worked. Watching the logs of such a run is genuinely strange: for the first thousand steps, little; then response lengths start climbing on their own — the model is *choosing* to think longer because thinking longer gets graded right more often — and somewhere along the way it begins re-reading its own work, backtracking, trying second approaches. Nobody wrote that behavior in a dataset. DeepSeek's R1-Zero made the recipe public and the pattern famous¹, and it did so with an algorithm, GRPO, whose defining feature is an *absence* — no critic network — and whose true cost lives somewhere no algorithm section will show you: in the fact that an RL step spends most of its wall-clock *generating text*, which is an inference workload, on a cluster you configured for training. This chapter covers both halves at full depth: the algorithm (§25.1–25.2) and the systems problem (§25.3) — because in RL, uniquely in this book, the systems half is the bigger bill.

ANATOMY OF ONE RL STEP (wall-clock, long-CoT regime — the numbers that reorganize your priorities)



The inversion to internalize: the box every paper discusses (update) is the small one. The big box is an INFERENCE workload — decode-bound, KV-cache-hungry, batch-elastic — running on hardware you configured for training. Chapter 13's serving economics and Chapter 14's parallelism both apply, but to different phases of the same step, over the same weights.

Consequence 1: every rollout-side efficiency (paged KV, continuous batching, TP-for-inference) is an RL training speed
Consequence 2: the engines want different weight layouts — the resharding problem §25.3 is about.

Figure 25.1 — Where the step actually goes. Generation dominates RL wall-clock the way attention dominated serving in Chapter 13 — and for the same reason: autoregressive decode. Optimizing the update loop before the rollout loop is the field's canonical misallocation.

25.1 Verifiable rewards: the reward is a function, not a model

Classic RLHF (Chapter 24) learns a reward model from human preferences and then optimizes against it — with the well-earned anxiety that the policy will find the reward model's blind spots and exploit them. The reasoning wave's founding simplification is to pick domains where *the reward can be computed*: a math answer either matches the reference or doesn't; code either passes the unit tests or doesn't; a constrained output either parses or doesn't. The reward becomes a rule — in R1's case literally two rules, an **accuracy check** (extract the boxed answer, compare) and a **format check** (thinking must live inside its tags)¹ — and the deepest failure mode of RLHF, hacking a learned grader, is off the table. Tulu 3 named the paradigm RLVR — reinforcement learning with verifiable rewards — and showed it working as a standard post-training stage beyond the frontier labs.²

What the simplification buys and costs, honestly. It buys *trustworthy signal at scale*: millions of gradings, zero annotation, no reward-model drift — and it is why Chapter 27's advice was to put your synthetic-data volume where verification is mechanical; the same logic, now driving training itself. It costs *coverage*: the recipe reaches only where checkers reach (math, code, logic, structured tasks), and everything else still needs Chapter 24's machinery. And it does not abolish hacking — it relocates it: with a binary grade, the model happily learns to game the *checker* (answer-extraction regexes, degenerate formats, guessing distributions), which is why the reward function deserves test cases like any other production code:

```
# a verifiable reward is production code – treat it like it
def reward_fn(data_source, solution_str, ground_truth, extra_info=None):
    """veRL's custom-reward signature: score one completed rollout."""
    answer = extract_boxed(solution_str)          # tolerant, but fuzz-tested:
    if answer is None:                             # the regex IS attack surface
        return 0.0
    ok = math_equal(answer, ground_truth)         # symbolic + numeric compare –
    fmt = has_valid_think_block(solution_str)     # "0.5" == "1/2" == "\frac{1}{2}"
    return 1.0 if (ok and fmt) else (0.1 if fmt else 0.0)
    # scale matters less than MONOTONICITY: never let a wrong answer with
    # pretty formatting outscore a right answer with ugly formatting.

# and the tests nobody writes until the model finds the hole for them:
assert reward_fn(_, "The answer is \boxed{042}", "42") == pytest.approx(1.0)
assert reward_fn(_, "\boxed{42} ... just kidding, \boxed{7}", "42") < 1.0
```

What a training sample actually looks like

Before the algorithm, look at the actual bytes — because "prompt" and "rollout" stay abstract until you have seen one, and half the practical bugs in this chapter (template mismatches, masking errors, extraction failures) are invisible at the abstract level. A training sample begins life as a dataset row, gets rendered through the model's **chat template** (Chapter 23's machinery — the special tokens must match the SFT stage exactly, or you are RL-ing a distribution the model has never seen), and comes back from the rollout engine with the thinking attached:

```
# ONE GRPO TRAINING SAMPLE, END TO END (Qwen-style template)
# -- dataset row (GSM8K) -----
{"question": "Natalia sold clips to 48 of her friends in April, and then
  she sold half as many clips in May. How many clips did she sell
  altogether in April and May?",
  "ground_truth": "72"}

# -- what the rollout engine is actually given (after chat template) ----
<|im_start|>system
You are a helpful assistant. Think step by step inside <think> tags,
then give the final answer as \boxed{...}<|im_end|>
<|im_start|>user
Natalia sold clips to 48 of her friends in April, ... altogether?<|im_end|>
<|im_start|>assistant
# ← generation starts HERE; everything above is prompt

# -- one of the G=16 rollouts that comes back -----
<think> April: 48 clips. May: half as many, so 48/2 = 24.
Total: 48 + 24 = 72. Let me double-check: half of 48 is 24, yes. </think>
The answer is \boxed{72}<|im_end|>

# -- what happens to it -----
reward_fn(...) → extract "72" → matches → r = 1.0      (§25.1)
loss mask      → prompt tokens: NO loss · all generated tokens: loss,
                weighted by this rollout's group advantage Â (§25.2)
```

And know the grading distribution you are optimizing against — the benchmarks are not interchangeable, and the difficulty band you train on (§25.8) should be chosen with them in view:

Table 25.1 — The benchmarks these runs live and die by. All graded by answer extraction + exact/symbolic match — the same machinery as the training reward, which is both convenient and a contamination risk (Ch. 9).

BENCHMARK	SHAPE	A REPRESENTATIVE ITEM	HOW IT'S SCORED
GSM8K	~1.3K grade-school word problems	the Natalia item above (answer: 72)	exact match on the final number; strong models saturate $\geq 95\%$ — a <i>training-band</i> source now, not a differentiator
MATH-500	500 competition problems, subset of MATH	"Let $f(x) = 3x + 2$, $g(x) = (f(x))^2 - 1$. Find $g(1)$." → 24	symbolic match ($0.5 = 1/2 = \frac{1}{2}$); mid-band difficulty
AIME 2024/25	30 items/year, integer answers 0–999	"Every morning Aya goes for a 9-kilometer walk..." (2024-I-1; answer: 204)	exact integer match; so small that pass@1 swings ± 3 pts on noise — report avg@k (§25.8)

Beyond math: when the checker is an executor

Math got the spotlight because its checker is a string comparison, but the second pillar of the verifiable regime is **code** — and code changes the reward's *physics*. A math reward is a regex and a symbolic compare: microseconds, on the driver, essentially free. A code reward means *running the model's program against test cases*, and the moment you write that sentence operationally, you have signed up for a second piece of infrastructure: a **sandboxed executor fleet**. Model-generated code is untrusted input by definition — it will, at RL scale, eventually contain infinite loops, fork bombs, gigabyte prints, attempts to read the reward file, and network calls, not out of malice but because you are sampling millions of programs from a distribution under optimization pressure — so every rollout's code runs in an isolated sandbox (containers, or microVMs at scale) with hard CPU, memory, and wall-clock limits, no network, and a kill switch. The reward that took microseconds now takes seconds, and it takes them *per test case per rollout*: a batch of $512 \text{ prompts} \times 16 \text{ rollouts} \times 10 \text{ tests}$ is 80K sandboxed executions per step. That is a CPU-fleet sizing problem sitting inside your reward function, and it moves the reward from a footnote in Figure 25.1 to a box you must actually budget — undersize the executor pool and grading becomes the straggler; veRL's reward workers exist as a separate, CPU-resident group for exactly this reason (§25.3).

The hacking surface also changes shape, in a way that rhymes with the fuzzing lesson above but is nastier. A math checker can be fuzzed to near-completeness; a *test suite cannot*. If your reward is "passes the visible tests," the model will learn to pass the visible tests — special-casing the sample inputs, hard-coding expected outputs, catching exceptions and printing the most common answer — because weak tests are a hackable reward exactly the way a learned model is, just with a smaller attack surface. The defenses are software-engineering defenses, not RL defenses: grade on *hidden* tests the model never conditions on, include adversarial and property-based cases (random inputs checked against a reference solution), and require passing *all* tests for reward rather than partial credit on easy ones. The quality of your test cases now bounds the quality of your policy — on our own runs we treat test-suite curation for code RL with the same seriousness as Chapter 10 treats pre-training data, because it *is* the data.

The third pillar is quieter but broader: **verifiable instruction following**. "Answer in exactly three sentences," "include the word *digital* twice," "output valid JSON matching this schema" — constraints a program can check with no executor and no math. IFEval catalogued 25 such instruction types precisely because they are objectively gradable³, and Tulu 3 folded them into RLVR alongside GSM8K as a standard target². This is the cheapest door into RLVR for a team without competition-math ambitions: the rewards are trivial to write, impossible to hack in interesting ways, and the capability — doing what the format asked — is commercially unglamorous and constantly demanded.

CONCEPT BOX · COLD START: RL FROM THE BASE, OR FROM SFT?

The first decision of an actual run is one this chapter's opening quietly skipped: *which model do you put on the table?* R1-Zero ran GRPO on the raw base — that was the scientific point, reasoning emerging from pure RL — and it worked, but the artifact was rough: chains of thought with poor readability, drifting between languages mid-derivation, formatted by accident¹. R1 proper reintroduced a small **cold-start SFT stage** — thousands of curated long-CoT examples (Chapter 23's machinery, tiny budget) — before the RL, and got faster convergence *and* readable output; every production recipe since has followed. The mechanism is worth stating because it guides the decision: RL can only reinforce behaviors the policy already samples with non-trivial probability. Cold-start SFT moves probability mass onto the *format* of good reasoning — think, then answer, in one language, in the template — so the sparse reward stops paying the exploration tax of discovering the format and

spends the whole budget improving the *content*. Run from the base when your goal is research on emergence; warm-start when your goal is a model — the cold-start set is days of work that buys back weeks of RL, and it is also where you set the language and style your users will actually read (for a Spanish-first model, this is where Spanish-language reasoning is made the default, not hoped for).

25.2 GRPO: the critic-shaped hole

PPO — the incumbent from RLHF — needs a **value model**: a second network, typically the policy's size, trained alongside it to estimate how good each state is, so that rewards can be converted into *advantages* ("was this better than expected?"). That critic is half your memory, half your engineering, and a chronic source of instability. GRPO's move, introduced for math RL in DeepSeekMath, is to notice that when you are already sampling several answers per prompt, *the group is the critic*:⁴ generate G rollouts for the same prompt, and score each against its groupmates —

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

— "better than the group average" is positive advantage, worse is negative, and no network had to learn anything. The full objective is then PPO's clipped surrogate with this advantage on every token of rollout i , plus a KL penalty to a frozen reference policy to keep the exploration anchored:

$$J = E \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_t \min \left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_i \right) \right] - \beta D_{KL} [\pi_\theta \| \pi_{\text{ref}}]$$

with $\rho_{i,t} = \pi_\theta(o_{i,t} \mid \cdot) / \pi_{\text{old}}(o_{i,t} \mid \cdot)$ the usual importance ratio. In code, the part that is actually GRPO fits in a screen:

```
# GRPO's core: group-normalized advantages + clipped update. No critic.
def grpo_advantages(rewards, group_size):
    r = rewards.view(-1, group_size)
    adv = (r - r.mean(dim=1, keepdim=True)) / (r.std(dim=1, keepdim=True) + 1e-6)
    return adv.flatten()

def grpo_loss(logp, logp_old, logp_ref, adv, mask, eps=0.2, beta=1e-3):
    ratio = (logp - logp_old).exp()
    clipped = torch.clamp(ratio, 1 - eps, 1 + eps)
    pg = -torch.min(ratio * adv, clipped * adv)
    kl = (logp_ref - logp).exp() - (logp_ref - logp) - 1
    per_tok = pg + beta * kl
    return (per_tok * mask).sum() / mask.sum()

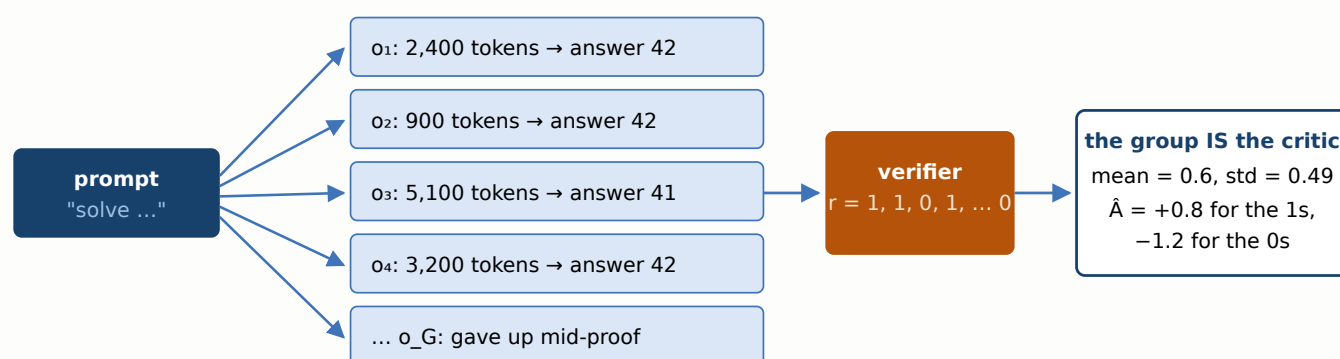
# refinements you will meet in the wild (one line each):
# Dr.GRPO: drop the /std normalization (length-bias fix)
# DAP0: raise the UPPER clip only ("clip-higher"), drop KL, and resample
# away all-correct/all-wrong groups (their advantage is zero – dead
```

```

signal)
# GSP0:    take the ratio at SEQUENCE level, not per token (MoE stability,
Qwen3)

```

Why it works so well in the verifiable regime is worth one honest paragraph, because GRPO is not magic — it is *matched to its setting*. Binary rewards make value estimation genuinely hard for a critic (the landscape is cliffs, not slopes) while making group statistics excellent (8–64 samples of right/wrong per prompt is a clean local baseline). The comparisons are within-prompt, so hard prompts don't drown easy ones. And deleting the critic halves the training-side memory precisely where §25.3 needs the room. The refinements in the code comment are the current frontier of tinkering — DAPO's open recipe, notably, hit 50 points on AIME 2024 with a 32B model in half the steps of the R1-style baseline by exactly those three tweaks⁵ — but the skeleton has been stable since DeepSeekMath: sample a group, normalize within it, clip, anchor.



No value network, no learned reward — a fan-out, a checker, and two group statistics. The deleted critic is half the training memory and most of the deleted instability; the price is needing G rollouts per prompt, which is why Figure 25.1's big box got bigger.

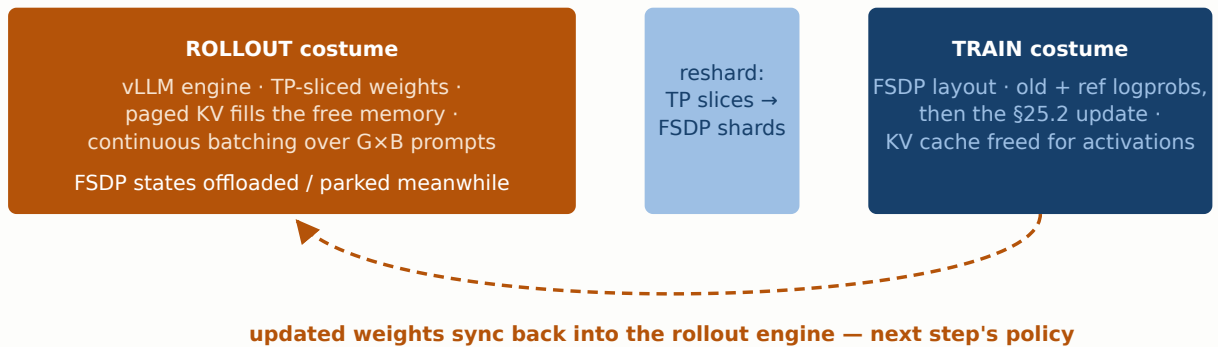
Figure 25.2 — GRPO in one picture. Sample a group, verify, normalize within the group, reinforce above-average rollouts. Note the trade flowing into the next section: the algorithm's simplicity was purchased with *more generation* — $G\times$ per prompt — making the systems problem $G\times$ more important.

25.3 Rollouts as the new bottleneck: the two-engine problem, and veRL

Now the systems half, and it begins where Chapter 14 left its forward pointer. Pre-training's parallelism serves *one* workload: forward-backward over fixed-shape batches. RL's step alternates *two* workloads with opposite personalities over the *same weights*. The **rollout phase** is inference: autoregressive decode, one token at a time, KV caches ballooning across thousands of variable-length sequences — it wants an inference engine (vLLM or SGLang: paged KV cache, continuous batching that refills as sequences finish, inference-grade TP⁶) and modest tensor parallelism. The **update phase** is training: it wants FSDP or Megatron sharding, the 16 bytes per parameter of Chapter 4, big fused batches. Neither engine's ideal layout is remotely like the other's — FSDP scatters flat shards across the world; vLLM wants contiguous TP slices — and the weights must be *the same weights*, refreshed every step, or your rollouts come from a stale policy and the importance ratios of §25.2 quietly stop meaning anything.

THE COLOCATED (HYBRID-ENGINE) STEP — same GPUs, two costumes

all GPUs →



The reshard is colocation's tax: the actor's parameters change layout twice per step. The 3D-HybridEngine makes it zero-redundancy (each GPU ships exactly the slices the other layout needs — seconds, even at 70B) and reuses the training GPUs' free memory as rollout KV space — why colocation beats separate pools at most cluster sizes.

Alternative — disaggregate: a rollout pool streams to a trainer pool; overlap gained, one-step staleness paid.

Figure 25.3 — Two costumes, one set of weights. Colocation (veRL's hybrid engine) time-slices the whole cluster between an inference engine and a trainer, paying a per-step reshard; disaggregation dedicates pools and pays in staleness. Below ~kilogpu scale, colocation is the pragmatic default — no GPUs idle waiting for the other phase.

veRL's answer, structurally. The framework (open-sourced from the HybridFlow paper⁷) resolves a tension that killed earlier RLHF systems: the *algorithm* is a dataflow you want to express in twenty lines (generate → score → logprobs → update), but each node of that dataflow is itself a distributed program spanning hundreds of GPUs. veRL splits the difference with a **single-controller / multi-controller hybrid**: one Ray driver process runs the algorithm as plain Python over worker-group handles — so a grad student can read the GRPO loop — while each worker group (actor, rollout, reference, reward) internally runs its own SPMD engine with whatever Chapter 14 mesh it needs (FSDP or Megatron for training, vLLM/SGLang TP for rollout). The **3D-HybridEngine** is the piece of hard engineering underneath: it colocates the actor's two costumes on the same resource pool and reshards weights between the training layout and the rollout layout each step with zero redundant transfers. Everything is declared, not programmed — the config is the systems design:

```
# veRL GRPO on 4 nodes: the systems design as hydra overrides (API per current docs)
python3 -m verl.trainer.main_ppo \
  algorithm.adv_estimator=grpo \
  data.train_files=$HOME/data/math_train.parquet \
  data.train_batch_size=512 \
  data.max_prompt_length=1024 data.max_response_length=8192 \
  actor_rollout_ref.model.path=Qwen/Qwen2.5-7B-Instruct \
  actor_rollout_ref.actor.strategy=fsdp \
  actor_rollout_ref.actor.ppo_mini_batch_size=64 \
  actor_rollout_ref.actor.use_kl_loss=True \
  actor_rollout_ref.actor.kl_loss_coef=0.001 \
  actor_rollout_ref.actor.loss_agg_mode=token-mean \
  actor_rollout_ref.rollout.name=vllm \
  actor_rollout_ref.rollout.n=16 \
  actor_rollout_ref.rollout.tensor_model_parallel_size=2 \
  actor_rollout_ref.rollout.gpu_memory_utilization=0.6 \
  custom_reward_function.path=./reward_fn.py \
```

G: the group size
KV vs parked FSDP
\$25.1's function

```

trainer.n_gpus_per_node=8 trainer.nnodes=4
# one command declares: the algorithm (GRPO), both engines and their meshes,
# the reshard boundary, and the reward. The driver loop it runs is ~a page.

```

Read three decisions out of that config the way §14.7 taught you to read a mesh. `rollout.n=16` is the GRPO group size — it multiplies Figure 25.1's big box, and it is the first knob to lower when steps crawl.

`rollout.tensor_model_parallel_size=2` is *inference* TP, chosen by vLLM's economics (fit the model + healthy KV space), fully decoupled from the training mesh — the hybrid engine reshards between them. And `gpu_memory_utilization=0.6` is the colocation treaty itself: how much of each GPU the rollout engine may claim for KV pages while the parked FSDP states hold the rest — set it too high and the reshard OOMs, too low and long-CoT batches thrash. Those three numbers, plus the §14.8 debugging discipline (an RL hang is still one missing rank — now it can be a rollout worker), are most of what operating GRPO day-to-day actually is.

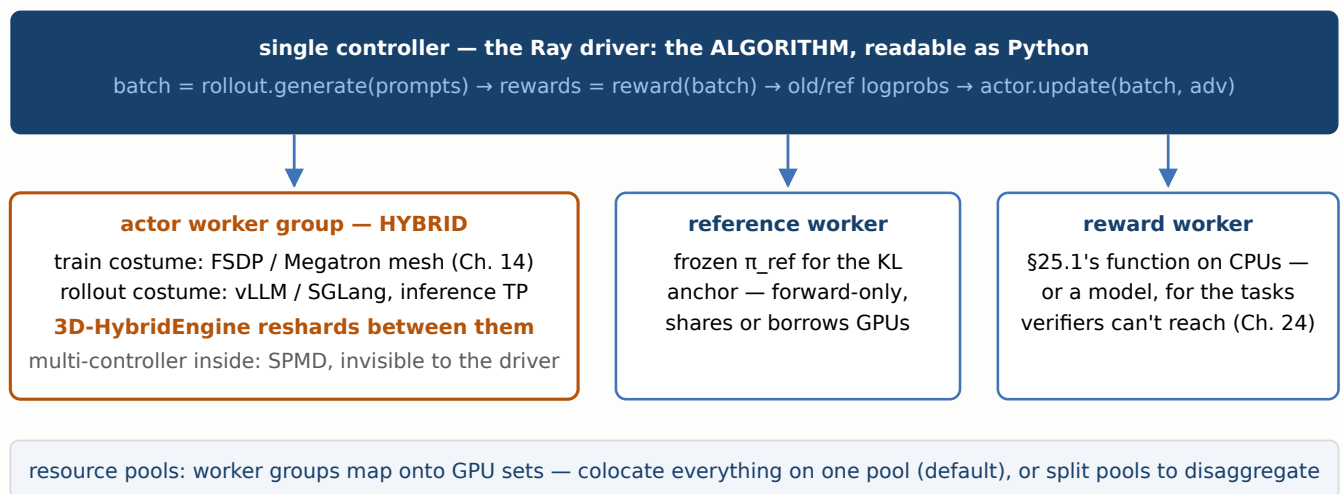


Figure 25.4 — veRL's shape. One process you can read (the driver) orchestrates worker groups you don't have to (each an SPMD engine with its own Chapter 14 mesh). The hybrid actor is the heart: two execution costumes over one set of weights, resharded by the 3D-HybridEngine each phase change.⁷

Async RL: paying staleness to escape the straggler tax

The colocated loop of Figure 25.3 has a flaw the diagrams above politely hide: it is **synchronous**, and a synchronous generation phase ends when its *longest* rollout ends. Reasoning rollouts have brutal length variance — in one group of 16, a model may answer in 400 tokens and ruminate for 12,000 — so for the last stretch of every generation phase, most of the cluster idles while a few stragglers finish their epics. The longer the thinking gets (and §25.4 said training makes it longer), the worse the tax: utilization decays over the run precisely because the run is working.

The escape is to break the phase barrier. **One-step overlap** is the gentle version: generate batch $k+1$ while training on batch k , so rollouts come from a policy exactly one update old — off-policy by a whisker, which §25.2's importance ratios were built to absorb, in exchange for hiding the training phase entirely. **Fully asynchronous** systems go further — AReaL is the clean public design⁸: a dedicated rollout fleet generates *continuously*, never waiting for a training step; finished (or even *interrupted-and-resumed*) trajectories stream into a buffer; the trainer consumes from the buffer at its own pace, tags every trajectory with the policy version that produced it, and the objective is corrected for the measured

staleness — with a hard cap (drop trajectories more than s versions old) so drift stays bounded. The reported effect is what the straggler analysis predicts: around 2–3× training throughput at matched quality on long-CoT math and code, with the gains growing as responses lengthen. The same machinery is what makes **agentic RL** feasible at all — episodes with tool calls and environment waits have unbounded, exogenous latency, and only a continuous-generation design can keep GPUs busy around them (veRL's agent-loop mode and its server-based rollout exist for exactly this).

THREE SCHEDULES (time →, one row of GPUs; navy = generate, orange = train, empty = idle)

synchronous



most rollouts done — cluster waits for the 12K-token straggler (thin bar)

one-step overlap



training (orange) rides under the NEXT batch's generation — rollouts are one version old; the ratios absorb it

fully async (AReaL-style)



every trajectory carries its policy version; the objective corrects for staleness; trajectories older than the cap are dropped

the trade, plainly: sync = pure on-policy, straggler-bound · one-step = ~free lunch for most setups · fully async = 2-3× throughput on long CoT, bought with bounded off-policy drift and a second system to operate

and the §25.8 pitfall stands doubled: in any async design, ASSERT the staleness bound in code — silent drift is the failure mode.

Figure 25.5 — Escaping the straggler. Synchronous steps end when the longest thinker finishes; one-step overlap hides training under the next generation; fully-async designs decouple the two entirely behind a version-tagged buffer with a staleness cap. Choose by response-length variance and cluster size — and re-read the §25.2 ratios as the mechanism that makes mild staleness mathematically licensed rather than merely tolerated.

25.4 Test-time compute in practice: budgeting the thinking

RL training and test-time compute (Chapter 7's third axis) are the same phenomenon seen from two sides: RL is how models *learn* to spend thinking tokens profitably, and the response-length growth in the opening is the model discovering the §7.3 curve on its own. Operationally that gift needs governing, in training and serving both. During training, cap `max_response_length` generously but handle what hits the cap deliberately — naively scoring truncated rollouts as failures teaches the model that long attempts are punished, a bias DAPO addresses by soft length penalties rather than hard zeroing⁵; and watch mean response length as a first-class training curve — healthy runs grow it in capability-linked waves, while runaway growth without accuracy gains means the model is being paid for effort, not results (usually a reward or KL misconfiguration). At serving, expose the budget: thinking-token limits per tier, the Chapter 7 economics of when a query deserves 10K tokens of deliberation, and the standing exit — once the expensive thinker exists, Chapter 26's distillation converts its behavior into cheap weights, which is why §25.6 is one paragraph long.

Budget is one axis of test-time compute: *how long* to think. The other is selection: think *several times* and keep the best — and "keep the best" needs a judge. That judge is where a whole tradition this chapter has so far ignored comes back through the side door.

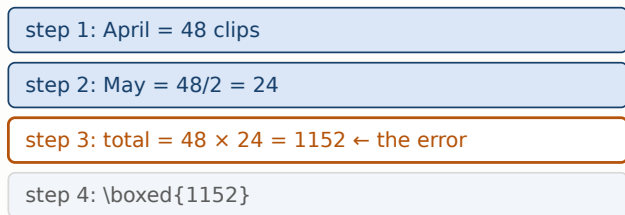
25.5 Learned verifiers: ORM, PRM, and why R1 walked away

The doctrine of §25.1 — the reward is a function, not a model — has an objection so natural your students will raise it in the first lecture. A binary outcome reward is *brutally sparse*: a 4,000-token derivation that goes wrong at step 14 of 30 earns exactly the same 0.0 as random noise, throwing away the information that half of it was excellent; and conversely a lucky guess with garbage reasoning earns the same 1.0 as a flawless proof. Surely, the objection goes, a reward that scored *each step* would assign credit where it belongs and learn faster. That intuition built a whole research tradition — and understanding why the tradition lost at training time, yet survives at test time, is one of the most instructive stories in modern post-training.

The two shapes of learned verifier are named by what they grade. An **outcome reward model** (ORM) reads a complete solution and emits one scalar: the probability the final answer is right. A **process reward model** (PRM) emits a score *per step*, so a solution's quality is a trajectory, not a verdict. The landmark comparison is OpenAI's "Let's Verify Step by Step": on competition math, with an identical generator, a PRM used as a best-of-N reranker solved **78.2%** of a MATH test subset at $N=1860$, against **72.4%** for an equivalently-trained ORM and **69.6%** for plain majority voting — and the gap *widened* with N^9 . Process supervision won decisively. But look at the price tag: the PRM was trained on **PRM800K**, roughly 800K step-level labels over 75K solutions to 12K problems, written by human annotators screened for 75% agreement with gold labels⁹. Per-step human annotation of mathematics is among the most expensive labeling work that exists, and it must be *redone* as the policy's error distribution shifts — the steps a strong model gets wrong are not the steps a weak model gets wrong.

The field's answer to the labeling bill was to make the labels themselves Monte-Carlo estimates. Math-Shepherd's construction is the one to know because it is fully automatic: to score step k of a solution, roll out $N=8$ completions *from that step* with a completer model and check their final answers against the reference. **Hard estimation** labels the step 1 if *any* completion reaches the right answer (the step is salvageable); **soft estimation** labels it with the *fraction* that do (the step's win probability) — empirically the two train equally good verifiers, so the cheaper hard label wins¹⁰. No human writes anything; the checker of §25.1 is bootstrapped into a step-level teacher. A Math-Shepherd PRM lifted Mistral-7B best-of-256 verification to 89.1% on GSM8K and 43.5% on MATH, and — used as a dense training reward in step-by-step PPO — pushed the same model from 77.9% to 84.1% on GSM8K¹⁰. Notice what the trick really is: it spends *generation compute* (8 completions per step) to buy *credit assignment* — the same currency-exchange this whole chapter keeps making, run in the opposite direction.

ONE ROLLOUT, TWO GRADERS — AND WHERE STEP LABELS COME FROM



ORM — one verdict, at the end

score = 0.08

the good steps 1–2 earn nothing;
WHERE it broke is invisible

PRM — a score per step

0.94

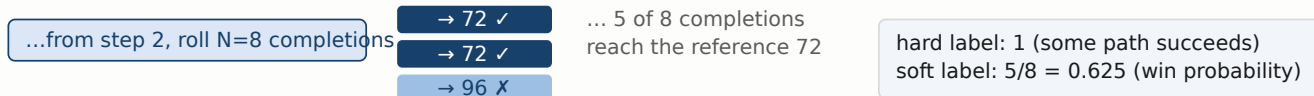
0.91

0.06

0.05

credit lands on step 3 — dense signal,
but per-step labels are required

WHERE THE LABELS COME FROM (Math-Shepherd: no humans)



the verdict of practice: as a TRAINING reward, the PRM is a learned model again — hackable under optimization pressure, stale as the policy moves, expensive to refresh (R1 dropped it for two rules).

as a TEST-TIME judge — best-of-N reranking, step-level search — selection pressure is mild, and it demonstrably pays.

Figure 25.6 — Two graders and a labeling trick. The ORM's single scalar wastes the structure of a near-miss; the PRM localizes the failure but needs step labels, which Math-Shepherd manufactures by Monte-Carlo: complete from each step and count successes. The bottom line is the section's thesis — dense learned rewards lost the training job to two rules, and kept the judging job.

Why R1 walked away. Given all that machinery worked, the most consequential sentence in the R1 report is a quiet one: its authors tried PRMs and MCTS-style search and shipped *neither*, listing the reasons as unsuccessful attempts¹. Three, for PRMs. First, *defining a step* is only clean in mathematics; in general reasoning, where one step ends and the next begins is a formatting convention, not a semantic fact. Second, the *labels*: model-annotated step labels (the Math-Shepherd route) proved unreliable at their scale, and human annotation cannot scale at all. Third — the deep one — a PRM in the training loop is a *learned reward model again*, and everything §25.1 escaped comes back: under 10^4 steps of optimization pressure the policy finds the PRM's blind spots and mines them, so the PRM must be retrained as the policy shifts, adding a training pipeline inside your training pipeline. MCTS fell to a related pair: language has no chess-like move set — the branching factor is the vocabulary — so search needs a value model to prune, and the value model is exactly as hard to train as the PRM was. Their conclusion draws precisely the line this section is built on: PRMs rerank well and search helps inference, but as the *training signal* at scale, the overhead and the hackability outweigh the density. Two rules beat a model. Sparse but incorruptible beat dense but gameable — under heavy optimization, the *integrity* of the signal matters more than its *information content*, which is as close to a moral as this book gets.

Where learned verifiers survive: selection. The test-time story is different because the pressure is different: a reranker choosing among N honest samples faces no gradient pointed at its own weaknesses. The menu, in ascending order of machinery. **Majority voting** (self-consistency) needs no verifier at all — sample N chains, extract answers, take the mode; it lifted GSM8K by +17.9 points on PaLM-540B in the original study¹¹ and remains the strongest baseline-per-line-of-code in the field, but it only exists where answers are voteable (math, multiple choice — not proofs, not code style). **Best-of-N with a verifier** replaces the mode with an argmax under an ORM/PRM — Lightman's 78.2% is this pattern — and works wherever the verifier does. **Step-level search** (beam search under a PRM, expanding only high-scoring prefixes) spends the same budget more surgically; the compute-optimal analysis shows it beating best-of-N at

small budgets on hard problems, while at large budgets the simpler methods catch up¹². All three are Chapter 7's test-time axis with a judge attached, and all three compose with the RL-trained thinker: o1-style long thinking *plus* verifier reranking is how the highest published competition-math numbers are assembled.

Two footnotes to the story complete the map. **Rejection-sampling fine-tuning** — generate with the checker, keep the survivors, SFT on them (STaR made it a loop; Llama 3 leaned on it heavily for its reasoning data¹³) — is what teams did before GRPO matured, and it is best understood as self-distillation with a verifier in the middle: Chapter 26's machinery, pointed at yourself. It remains the right tool when your rollout infrastructure can't do RL but can do batch generation. And **when does a PRM still earn its keep in training?** Where no final-answer checker exists but steps are locally checkable — theorem-proving outside formal systems, multi-hop agent episodes where partial credit for reaching the right sub-goal is the only signal available. There the choice is not "PRM vs. rule" but "PRM vs. nothing," and the R1 objections soften to engineering cautions: cap the optimization pressure, refresh the PRM on-policy, and audit for hacking like it owes you money — because it does.

25.6 Distilling reasoning traces (→ Chapter 26)

The full treatment lives in Chapter 26, but the placement deserves restating from this side of the fence, because it is the most consequential efficiency fact in this chapter: **for building small reasoners, distilling a strong RL-trained teacher's traces beats running RL on the small model directly** — 72.6 vs 47.0 on AIME for the same 32B student, at a fraction of the cost, in R1's own published comparison.¹ The economical pattern for a model family is therefore: pay this chapter's bill *once*, on the strongest base you have; harvest verified traces; SFT them into every smaller tier (Ch. 26); and reserve small-model RL for a final polish where evals say it still pays. RL discovers; distillation distributes.

25.7 What the big labs do

Table 25.2 — Reasoning RL, as shipped.

SYSTEM	RECIPE	THE LESSON
DeepSeek R1 / R1-Zero ¹	GRPO, rule rewards (accuracy + format), no critic, no PRM (§25.5); then SFT-warmed multi-stage pipeline for R1 proper	The public existence proof — and R1-Zero's readability problems are why the SFT warm start returned
DAPO (open recipe) ⁵	GRPO + clip-higher, no KL, dynamic sampling of dead groups, token-level loss, soft length penalty	50 AIME points on Qwen2.5-32B in half the baseline steps — the tweaks are worth real money
Tülu 3 (RLVR) ²	verifiable rewards as a standard post-training stage on open models	The paradigm works outside frontier labs, at academic budgets
Qwen3 (GSPO) ¹⁴	sequence-level importance ratios replacing per-token	Long-CoT + MoE destabilize token ratios; the objective is still moving

SYSTEM	RECIPE	THE LESSON
Kimi k1.5 ¹⁵	long-CoT RL with explicit length control and curriculum	Length is a managed resource, not a free variable
veRL ecosystem ⁷	hybrid-engine GRPO/PPO as open infrastructure (FSDP/Megatron × vLLM/SGLang)	The systems half is now a library — your team's leverage is the reward function and the data

25.8 Practical considerations

Watch four curves, not one. Reward-per-step is the vanity metric; the diagnosis lives in the ensemble: *mean response length* (§25.4's waves vs. runaway), *policy entropy* (the next paragraph — the most theoretically loaded of the four), *fraction of dead groups* (all-correct or all-wrong = zero advantage = paid generation, no learning; DAPO's dynamic sampling exists for this — and its growth means your data's difficulty no longer matches the model), and *KL to reference* (a slow leash-break is drift; a spike is a bug). Any one of them alone will lie to you for days.

Entropy collapse is the signature failure, and it is not noise. The pattern: pass@1 climbs, everything looks healthy, but the policy's entropy decays monotonically toward zero — and with it goes pass@k, because the model now writes the *same* solution k times. Exploration has died while exploitation still reports gains, and when entropy exhausts, the run flatlines at a ceiling you could have predicted weeks earlier: empirically, downstream performance trades against entropy along a clean exponential $R = -a e^H + b$, so the entropy curve is a forecast of where the run stops¹⁶. The mechanism is mechanical, not mysterious: policy-gradient updates change a token's logit in proportion to (probability × advantage), so tokens that are *already likely and currently winning* get pushed up hardest — a positive probability-advantage covariance that sharpens the distribution at every step, which is why entropy falls by default and never recovers on its own¹⁶. The fixes all target the same covariance from different angles: DAPO's **clip-higher** raises the upper clip bound so that *unlikely* tokens with positive advantage — the exploratory ones — can actually grow instead of being clipped away⁵; Clip-Cov and KL-Cov go after the culprits directly, clipping or KL-penalizing updates on the highest-covariance tokens¹⁶; and the blunt instruments (entropy bonuses, higher sampling temperature) trade calibration for diversity. Operationally: alarm on the entropy *slope*, not its level, and treat a pass@1-up/pass@k-down divergence as the same alarm wearing eval clothing — by the time pass@1 stalls, the recoverable moment has usually passed.

The data is a difficulty curriculum whether you curate it or not. GRPO learns nothing from prompts the model always solves or never solves; the signal lives at the frontier of current ability. Filter the prompt pool to a target solve-rate band (roughly 20–80% under the current policy, re-estimated as it improves) and the same GPU-hours buy several times the learning — the highest-leverage lever in the whole chapter, and it is data work, not systems work.

Evaluate with variance discipline. Pass@1 on a 30-problem competition set moves ±3 points on sampling noise alone; report avg@k over many seeds, fixed decoding settings, and Chapter 9's contamination hygiene — reasoning benchmarks are the most leaked-into category there is. Half of the "RL gains" that fail to replicate were measurement.

Common pitfalls

Checker-hacking.

The model learns your answer-extraction regex, not arithmetic. Fuzz the reward function (§25.1) and read raw rollouts weekly — greps don't catch what eyes do.

Truncation bias.

Hard-failing capped rollouts teaches "long = bad." Soft penalties, or exclude truncated samples from the loss entirely.

Stale-policy rollouts.

A broken weight sync (or too-aggressive async) quietly breaks the importance ratios; loss looks fine, learning stalls. Assert sync freshness in the loop.

Rollout OOM at hour 30.

Response lengths grow during training — the KV budget that fit at step 0 doesn't at step 500. Budget `gpu_memory_utilization` for the lengths you'll reach, not the ones you start with.

The reference model bill.

π_{ref} adds a forward pass over every token of every rollout. Share its GPUs, cache its logprobs with the rollout pass, or (DAPO-style) drop the KL and monitor drift by eval instead.

RL before its prerequisites.

On a weak base, sparse rewards find nothing to reinforce (Chapter 5's lesson: capability comes from earlier stages). Distill first, RL last — §25.6's ordering is not optional at small scale.

25.9 Decision framework

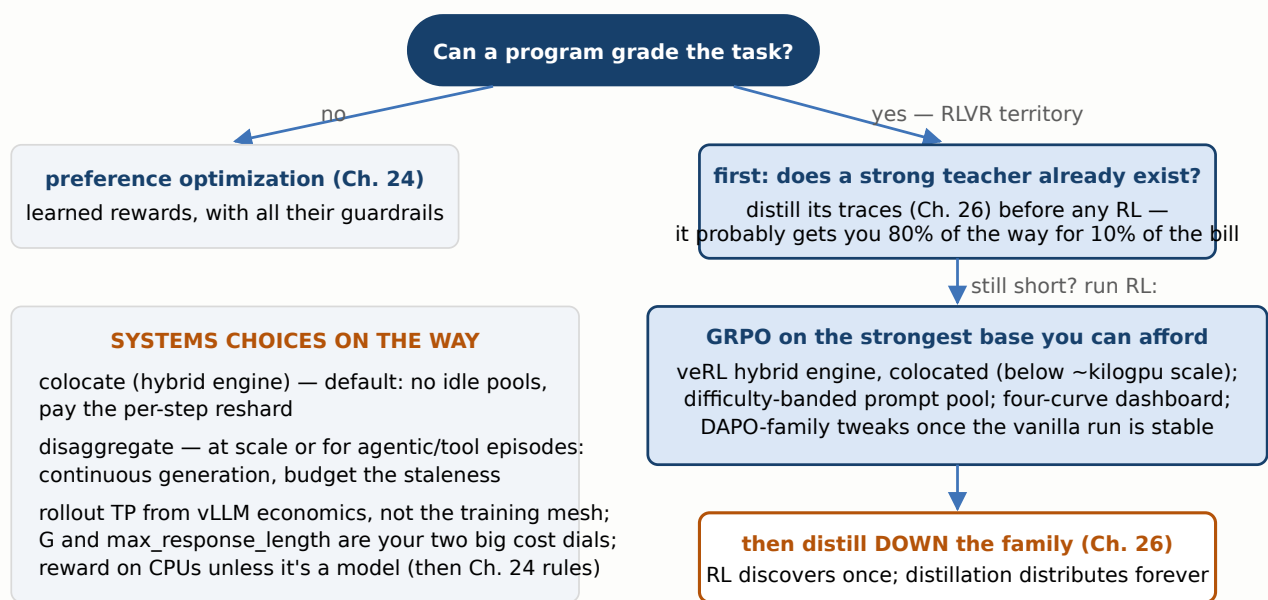


Figure 25.7 — Choosing your way into reasoning RL. The verifier gates entry; distillation brackets the RL on both sides — as the cheaper substitute before, and the distribution mechanism after. The systems box is \$25.3 compressed to its three dials.

Cost-benefit, bluntly. A verifiable-reward function: days of careful work, and it is the whole foundation — budget fuzzing time or pay in hacked rewards. GRPO over PPO: strictly cheaper (no critic) in the verifiable regime — the default without hesitation. veRL over hand-rolling: weeks of saved engineering and a maintained reshard path; your differentiation was never going to be the trainer loop. The RL run itself: the most expensive per-point-of-eval stage in post-training (Figure 25.1's big box, times G), justified when a verifier exists, the base is strong, and the capability matters commercially — and its output *appreciates*, because the traces feed Chapter 26. **How this scales:** below ~16 GPUs, skip straight to distillation; at 16–256, colocated GRPO with a banded prompt pool is the sweet spot this book's audience lives in; at frontier scale the game shifts to asynchronous pipelines and environment engineering — and the four curves on the dashboard stay exactly the same.

25.10 Summary

Reasoning RL rests on one substitution and one deletion. The substitution: learned reward models give way to *verifiable rewards* — programs that grade math, code, and structure — trading coverage for incorruptible signal at scale, and relocating vigilance to the checker itself (which, for code, is a sandboxed executor fleet with a test suite whose quality bounds the policy's). The learned-verifier tradition — ORMs, PRMs, and their Monte-Carlo-labeled descendants — lost the training job to that substitution for a reason worth memorizing (under heavy optimization, signal integrity beats signal density) and kept the selection job, where best-of- N and step-search under a verifier remain how the biggest test-time numbers are assembled. The deletion: GRPO removes PPO's critic and lets the group be the baseline — sample G rollouts per prompt, normalize rewards within the group, clip, anchor with a KL to reference — a skeleton stable since DeepSeekMath, refined at the margins (clip-higher, dead-group resampling, sequence-level ratios) where the wild now

tinkers. The true cost structure is systemic: 70–90% of the step is *generation*, an inference workload interleaved with a training workload over the same weights, and the craft is the two-engine choreography — vLLM's paged, continuously-batched decode in one costume, FSDP/Megatron in the other, with veRL's hybrid engine resharding between them each phase and a one-page driver loop expressing the algorithm. Operate by the four curves (length, entropy, dead groups, KL), feed a difficulty-banded prompt pool, fuzz the reward like production code, and hold the family strategy that brackets this whole chapter: distill before RL where a teacher exists, RL once on the strongest base, distill the discoveries down — because RL is how capability is found, and Chapter 26 is how it is delivered.

References

1. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. R1-Zero's pure-RL emergence (length growth, re-evaluation behaviors), rule-based accuracy+format rewards, and the distill-vs-direct-RL comparison.
2. Lambert, N., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. RLVR as a standard, reproducible post-training stage.
3. Zhou, J., et al. (2023). Instruction-Following Evaluation for Large Language Models (IFEval). arXiv:2311.07911. 25 types of programmatically verifiable instructions — the catalogue that verifiable instruction-following rewards are built from.
4. Shao, Z., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300. GRPO: group-normalized advantages replacing the value network.
5. Yu, Q., et al. (2025). DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476. Clip-higher, dynamic sampling, token-level loss, soft overlong punishment; 50 AIME points on Qwen2.5-32B at half the steps.
6. Kwon, W., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM). SOSP. arXiv:2309.06180. The rollout engine's two load-bearing ideas: paged KV and continuous batching.
7. Sheng, G., et al. (2025). HybridFlow: A Flexible and Efficient RLHF Framework (veRL). EuroSys. arXiv:2409.19256. Single-controller/multi-controller hybrid; the 3D-HybridEngine's zero-redundancy actor resharding between training and rollout layouts.
8. Fu, W., et al. (2025). AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. arXiv:2505.24298. Continuous, interruptible rollout generation; version-tagged trajectories with staleness-aware objective and a hard staleness cap; $\sim 2\text{--}3\times$ throughput at matched quality on long-CoT tasks.
9. Lightman, H., et al. (2023). Let's Verify Step by Step. arXiv:2305.20050. PRM vs. ORM at scale: 78.2% vs. 72.4% vs. 69.6% (majority) best-of-1860 on a MATH subset; the PRM800K step-label dataset ($\sim 800\text{K}$ labels, 75K solutions, 12K problems).
10. Wang, P., et al. (2024). Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. ACL. arXiv:2312.08935. Monte-Carlo step labeling ($N=8$ completions per step; hard vs. soft estimation); best-of-256 verification 89.1% GSM8K / 43.5% MATH; step-by-step PPO lifting Mistral-7B 77.9% $\rightarrow$ 84.1% on GSM8K.

11. Wang, X., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*. arXiv:2203.11171. Majority voting over sampled chains: +17.9 points on GSM8K with PaLM-540B.
12. Snell, C., et al. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. arXiv:2408.03314. Compute-optimal comparison of best-of-N vs. PRM-guided search vs. revisions across difficulty bands.
13. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Rejection sampling with verifier filtering as a core post-training data engine for reasoning.
14. Zheng, C., et al. (2025). Group Sequence Policy Optimization (GSPO). arXiv:2507.18071. Sequence-level importance ratios; the stability fix behind Qwen3's RL.
15. Kimi Team (2025). Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599. Long-CoT RL with explicit length control and curriculum.
16. Cui, G., et al. (2025). The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. arXiv:2505.22617. The $R = -ae^H + b$ performance–entropy trade; covariance-driven entropy decay; Clip-Cov and KL-Cov.

Related: *Chapter 14 — Distributing the model* (the meshes both engines run on) · *Chapter 26 — Distillation* (the cheaper substitute before, the delivery mechanism after) · *Chapter 7 — scaling laws* (the test-time axis this chapter trains) · *Chapter 24 — preferences* (when no verifier exists) · *Chapter 29 — the case study*. · [Training Efficient LLMs](#)